

Sprint 8 Structured Notes: Objects, Methods, JSON, and Safe Property Access

0. What Sprint 8 Is About

Sprint 8 is about **structured data**.

In earlier sprints, you learned how to work with:

- values
- variables
- strings
- numbers
- booleans
- conditions
- functions
- arrays
- loops

Now Sprint 8 teaches you how to group related data together using **objects**.

A real app usually does not only store single values like this:

```
const name = "Alice";  
const age = 25;  
const isActive = true;
```

A real app often stores data as one connected structure:

```
const user = {  
  name: "Alice",  
  age: 25,  
  isActive: true  
};
```

That is the main idea of Sprint 8.

Main goal

Represent structured data and access it safely.

By the end of Sprint 8, you should understand

- objects
 - properties
 - key-value pairs
 - dot notation
 - bracket notation
 - dynamic property access
 - adding object properties
 - updating object properties
 - deleting object properties
 - creating a new object without a property
 - checking if a property exists
 - `Object.hasOwnProperty()`
 - `hasOwnProperty()`
 - the `in` operator
 - why checking with `if (object.property)` can be wrong
 - nested objects
 - arrays inside objects
 - arrays of objects
 - looping through objects with `for...in`
 - primitive vs non-primitive values
 - object references
 - `const` with objects
 - methods
 - `this` inside beginner object methods
 - the `Object()` constructor
 - why object literal syntax is better for beginners
 - what happens with `Object(null)` and `Object(undefined)`
 - JSON
 - JSON syntax rules
 - `JSON.stringify()`
 - `JSON.parse()`
 - `JSON.stringify()` spacing
 - optional chaining `?.`
 - object destructuring
 - renaming during destructuring
 - default values during destructuring
 - nested destructuring
 - object rest syntax
 - object property shorthand
 - common beginner mistakes with objects
-

Part A: Current Progress Before Sprint 8

Sprint 8 depends on the earlier sprints.

Earlier Sprint	What You Learned	How It Helps With Sprint 8
Sprint 1	JavaScript can make a page respond to actions.	Objects can store the data that the page uses.
Sprint 2	Values, types, variables, <code>let</code> , <code>const</code> , and references.	Objects are values, and they are copied by reference.
Sprint 3	Strings, indexes, and text methods.	Object properties often store strings.
Sprint 4	Numbers, booleans, comparisons, conditions, and truthy/falsy values.	Object properties often store numbers and booleans. Property checks need careful conditions.
Sprint 5	Functions, return values, scope, and callbacks.	Object methods are functions stored inside objects.
Sprint 6	Arrays, indexes, nested arrays, and shallow copies.	Objects can contain arrays, and arrays can contain objects.
Sprint 7	Loops, <code>for...of</code> , <code>for...in</code> , and repeated processing.	<code>for...in</code> is useful for looping through object keys.

Simple summary:

Before Sprint 8, you learned how to store and process values. Sprint 8 teaches you how to organize related values into structured data.

Part B: The Core Idea of Objects

1. What Is an Object?

An **object** is a value that stores related data together.

An object uses **key-value pairs**.

Example:

```
const user = {  
  name: "Alice",
```

```
    age: 25,  
    active: true  
};
```

This object has three pieces of data:

```
name    -> "Alice"  
age     -> 25  
active  -> true
```

Simple meaning:

An object is a container for related information.

Objects are useful for things like:

- users
- students
- products
- books
- tasks
- settings
- profiles
- shopping carts
- game characters
- form data

2. Why Objects Are Useful

Without an object, related values can become disconnected.

Less organized:

```
const studentName = "Sangeeth";  
const studentAge = 16;  
const studentCourse = "JavaScript";  
const studentActive = true;
```

Better organized:

```
const student = {  
  name: "Sangeeth",
```

```
    age: 16,  
    course: "JavaScript",  
    active: true  
};
```

Now all the student data belongs to one structure.

Simple meaning:

Objects help you keep related data in one place.

3. Object Syntax

Objects usually use curly braces:

```
const objectName = {  
  key: value,  
  key: value  
};
```

Example:

```
const book = {  
  title: "JavaScript Basics",  
  pages: 200,  
  finished: false  
};
```

Important parts:

Part	Meaning
<code>const book</code>	Create a variable named <code>book</code> .
<code>{ ... }</code>	Create an object.
<code>title</code>	A property name, also called a key.
<code>"JavaScript Basics"</code>	The value stored in the <code>title</code> property.
<code>:</code>	Separates the key from the value.
<code>,</code>	Separates properties.

4. Properties and Key-Value Pairs

A property is one key-value pair inside an object.

Example:

```
const user = {  
  name: "Alice"  
};
```

This property is:

```
name: "Alice"
```

- `name` is the key.
- `"Alice"` is the value.

Simple meaning:

A property is one named piece of data inside an object.

5. Object Values Can Have Many Types

Object property values can be many different types.

```
const example = {  
  text: "hello",  
  number: 42,  
  boolean: true,  
  empty: null,  
  missing: undefined,  
  list: ["HTML", "CSS", "JavaScript"],  
  nestedObject: {  
    level: "beginner"  
  },  
  sayHi: function() {  
    return "Hi";  
  }  
};
```

This object contains:

- a string
- a number
- a boolean
- `null`
- `undefined`
- an array
- another object
- a function

Simple meaning:

An object can store almost any kind of JavaScript value.

Part C: Reading Object Properties

There are two main ways to read object properties:

1. dot notation
 2. bracket notation
-

6. Dot Notation

Use **dot notation** when the property name is simple and known.

Example:

```
const student = {  
  name: "Sangeeth",  
  course: "JavaScript"  
};  
  
console.log(student.name);  
console.log(student.course);
```

Expected output:

```
Sangeeth  
JavaScript
```

Read this:

```
student.name
```

as:

Get the `name` property from the `student` object.

Beginner rule

Use dot notation when the property name is normal and written directly in your code.

Good use of dot notation:

```
user.name  
book.title  
product.price  
student.score
```

7. Bracket Notation

Use **bracket notation** when:

- the property name has spaces
- the property name has special characters
- the property name is stored in a variable
- the property name is dynamic

Example with a space in the key:

```
const student = {  
  name: "Sangeeth",  
  "favorite language": "JavaScript"  
};  
  
console.log(student["favorite language"]);
```

Expected output:

```
JavaScript
```


This does not work:

```
console.log(student.favorite language); // SyntaxError
```

Why?

Because JavaScript sees the space and does not understand the property name.

Beginner rule

Use bracket notation when dot notation cannot clearly represent the property name.

8. Dynamic Property Access

Dynamic property access means the property name is stored in a variable.

Example:

```
const student = {  
  name: "Sangeeth",  
  age: 16,  
  course: "JavaScript"  
};  
  
const key = "course";  
  
console.log(student[key]);
```

Expected output:

```
JavaScript
```

This line:

```
student[key]
```

means:

Look inside the variable `key`, get its value, and use that value as the property name.

Since:

```
const key = "course";
```

This:

```
student[key]
```

becomes like this:

```
student["course"]
```

So JavaScript returns:

```
"JavaScript"
```

9. Dot Notation vs Bracket Notation

This distinction is important.

```
const student = {  
  name: "Sangeeth",  
  course: "JavaScript"  
};  
  
const key = "course";  
  
console.log(student.key);  
console.log(student[key]);
```

Expected output:

```
undefined  
JavaScript
```

Why?

```
student.key
```

means:

Look for a property literally named `key`.

But the object does not have a `key` property.

```
student[key]
```

means:

Use the value inside the `key` variable as the property name.

Since `key` contains `"course"`, JavaScript reads `student.course`.

Summary

Syntax	Meaning
<code>object.property</code>	Use the exact property name written after the dot.
<code>object[variable]</code>	Use the value stored inside the variable as the property name.
<code>object["property"]</code>	Use the string as the property name.

Part D: Adding, Updating, and Deleting Properties

10. Adding a New Property

You can add a property after the object is created.

```
const user = {  
  name: "Alice"  
};  
  
user.active = true;  
  
console.log(user);
```

Expected output:

```
{ name: "Alice", active: true }
```

This line adds a new property:

```
user.active = true;
```

Simple meaning:

Add a property named `active` and store `true` inside it.

11. Updating an Existing Property

You can update a property by assigning a new value.

```
const user = {  
  name: "Alice",  
  score: 10  
};  
  
user.score = 20;  
  
console.log(user.score);
```

Expected output:

```
20
```

This line changes the old value:

```
user.score = 20;
```

Simple meaning:

Replace the old `score` value with `20`.

12. Adding and Updating With Bracket Notation

You can also add and update properties with bracket notation.

```
const user = {  
  name: "Alice"  
};  
  
user["score"] = 95;  
user["name"] = "Ada";  
  
console.log(user);
```

Expected output:

```
{ name: "Ada", score: 95 }
```

Bracket notation is useful when the property name is dynamic:

```
const user = {};  
const key = "email";  
  
user[key] = "alice@example.com";  
  
console.log(user.email);
```

Expected output:

```
alice@example.com
```

13. Deleting a Property With `delete`

The `delete` operator removes a property from an object.

```
const user = {  
  name: "Alice",  
  password: "secret123"  
};  
  
delete user.password;  
  
console.log(user);
```

Expected output:

```
{ name: "Alice" }
```

Simple meaning:

Remove the `password` property from the object.

Beginner warning

`delete` changes the original object.

That is sometimes fine, but in many codebases, developers prefer creating a new object instead of changing the original one.

14. Creating a New Object Without a Property

You can use destructuring with rest syntax to create a new object without one property.

```
const user = {  
  name: "Alice",  
  password: "secret123",  
  role: "student"  
};  
  
const { password, ...safeUser } = user;  
  
console.log(safeUser);
```

Expected output:

```
{ name: "Alice", role: "student" }
```

This line:

```
const { password, ...safeUser } = user;
```

means:

Take `password` out. Put the rest of the properties into a new object called `safeUser`.

Mental model

Original object:

```
{
  name: "Alice",
  password: "secret123",
  role: "student"
}
```

New object:

```
{
  name: "Alice",
  role: "student"
}
```

This is useful when you want to remove sensitive or unnecessary data before using an object somewhere else.

Part E: Checking If a Property Exists

Checking object properties carefully is one of the most important Sprint 8 skills.

15. Why Simple Truthy/Falsy Checks Can Be Wrong

A common beginner mistake is this:

```
if (user.score) {
  console.log("Score exists");
}
```

This looks reasonable, but it can be wrong.

Example:

```
const user = {
  score: 0
};
```

```
if (user.score) {  
  console.log("Score exists");  
} else {  
  console.log("Score missing");  
}
```

Expected output:

Score missing

But the score is not missing.

The score exists and its value is `0`.

The problem is:

`0`

is a falsy value in JavaScript.

Falsy values include:

```
false  
0  
""  
null  
undefined  
NaN
```

So this check:

```
if (user.score)
```

actually asks:

Is `user.score` truthy?

It does not ask:

Does the `score` property exist?

Beginner rule

Do not use truthy/falsy checks when `0`, `false`, or `""` could be valid values.

16. Checking With `Object.hasOwn()`

`Object.hasOwn()` checks whether an object directly has a property.

Example:

```
const user = {  
  score: 0  
};  
  
console.log(Object.hasOwn(user, "score"));
```

Expected output:

```
true
```

This works even though the value is `0`.

Simple meaning:

Does this object directly contain a property named `score`?

Another example:

```
const user = {  
  name: "Alice",  
  active: false  
};  
  
console.log(Object.hasOwn(user, "name"));  
console.log(Object.hasOwn(user, "active"));  
console.log(Object.hasOwn(user, "email"));
```

Expected output:

```
true
true
false
```

Why is `active` true?

Because the property exists, even though its value is `false`.

Beginner rule

Use `Object.hasOwn(object, "property")` when you care about whether a property exists.

17. Checking With `hasOwnProperty()`

Objects can also use `hasOwnProperty()`.

```
const user = {
  name: "Alice"
};

console.log(user.hasOwnProperty("name"));
console.log(user.hasOwnProperty("email"));
```

Expected output:

```
true
false
```

This method also checks whether the object directly owns the property.

However, beginner-friendly modern code can prefer:

```
Object.hasOwn(user, "name")
```

because it avoids some edge cases with unusual objects.

18. Checking With the `in` Operator

The `in` operator checks whether a property exists in an object.

```
const user = {
  name: "Alice",
  score: 0
};

console.log("name" in user);
console.log("score" in user);
console.log("email" in user);
```

Expected output:

```
true
true
false
```

Important detail:

`in` checks more than direct properties. It can also find inherited properties from the prototype chain.

For beginner object checks, this is usually simpler:

```
Object.hasOwn(user, "score")
```

Simple comparison

Check	What It Checks	Beginner Recommendation
<code>if (user.score)</code>	Whether the value is truthy	Risky when <code>0</code> , <code>false</code> , or <code>""</code> are valid
<code>Object.hasOwn(user, "score")</code>	Whether the object directly owns the property	Good default
<code>user.hasOwnProperty("score")</code>	Whether the object directly owns the property	Common, but less robust than <code>Object.hasOwn()</code>
<code>"score" in user</code>	Whether the property exists directly or through inheritance	Useful, but understand it checks inherited properties too

19. Checking by Comparing With `undefined`

You may see this:

```
if (user.score !== undefined) {  
  console.log("Score exists");  
}
```

This works in many simple cases, but it has a weakness.

Example:

```
const user = {  
  score: undefined  
};  
  
console.log(user.score !== undefined);  
console.log(Object.hasOwn(user, "score"));
```

Expected output:

```
false  
true
```

The property exists, but its value is `undefined`.

So comparing with `undefined` checks the value, not the existence of the property.

Beginner rule

If you really need to know whether the property exists, use:

```
Object.hasOwn(object, "propertyName")
```

Part F: Nested Objects and Arrays

Real data is often nested.

Nested means:

One structure is inside another structure.

20. Nested Objects

An object can contain another object.

```
const person = {  
  name: "Alice",  
  contact: {  
    email: "alice@example.com",  
    phone: {  
      work: "098-765-4321"  
    }  
  }  
};  
  
console.log(person.contact.email);  
console.log(person.contact.phone.work);
```

Expected output:

```
alice@example.com  
098-765-4321
```

Read this:

```
person.contact.phone.work
```

as:

Start at `person`, go into `contact`, then go into `phone`, then get `work`.

21. Objects With Arrays Inside

Objects can contain arrays.

```
const student = {  
  name: "Sangeeth",  
  courses: ["HTML", "CSS", "JavaScript"]  
};
```

```
console.log(student.courses[0]);  
console.log(student.courses[2]);
```

Expected output:

```
HTML  
JavaScript
```

Read this:

```
student.courses[2]
```

as:

Get the `courses` array from `student`, then get the item at index `2`.

This combines:

- object property access
- array indexing

22. Arrays of Objects

An array can contain objects.

This is one of the most important beginner patterns.

```
const students = [  
  { name: "Ada", score: 95 },  
  { name: "Grace", score: 88 },  
  { name: "Linus", score: 76 }  
];  
  
console.log(students[0].name);  
console.log(students[1].score);
```

Expected output:

```
Ada  
88
```

Read this:

```
students[1].score
```

as:

Get item at index `1` from the `students` array, then get its `score` property.

The item at index `1` is:

```
{ name: "Grace", score: 88 }
```

So:

```
students[1].score
```

is:

```
88
```

23. Nested Arrays and Objects Together

Objects and arrays are often mixed.

```
const course = {  
  title: "JavaScript",  
  students: [  
    {  
      name: "Ada",  
      scores: [90, 95, 100]  
    },  
    {  
      name: "Grace",  
      scores: [85, 88, 92]  
    }  
  ]  
}
```

```
    ]  
  };  
  
  console.log(course.students[0].name);  
  console.log(course.students[1].scores[2]);
```

Expected output:

```
Ada  
92
```

Read this:

```
course.students[1].scores[2]
```

step by step:

1. `course` is the main object.
2. `course.students` gets the students array.
3. `course.students[1]` gets the second student object.
4. `course.students[1].scores` gets that student's scores array.
5. `course.students[1].scores[2]` gets the third score.

Part G: Looping Through Objects

24. `for...in` Loops Through Object Keys

Use `for...in` to loop through object keys.

```
const user = {  
  name: "Alice",  
  active: true,  
  score: 95  
};  
  
for (const key in user) {  
  console.log(key);  
}
```

Expected output:


```
name
active
score
```

Simple meaning:

For each key in the `user` object, run the loop body.

25. Getting Values With `for...in`

`for...in` gives you keys.

To get values, use bracket notation.

```
const user = {
  name: "Alice",
  active: true,
  score: 95
};

for (const key in user) {
  console.log(key, user[key]);
}
```

Expected output:

```
name Alice
active true
score 95
```

Why use bracket notation?

Because `key` changes during each loop.

Dry run:

Loop Step	key	user[key]
1	"name"	"Alice"
2	"active"	true

Loop Step	key	user[key]
3	"score"	95

This does not work for dynamic keys:

```
user.key
```

because it looks for a property literally named `key`.

26. Looping Through an Array of Objects

Use `for...of` when looping through array values.

```
const students = [
  { name: "Ada", score: 95 },
  { name: "Grace", score: 88 },
  { name: "Linus", score: 76 }
];

for (const student of students) {
  console.log(student.name, student.score);
}
```

Expected output:

```
Ada 95
Grace 88
Linus 76
```

Simple meaning:

For each student object inside the students array, print the student's name and score.

Beginner rule

- Use `for...in` for object keys.
- Use `for...of` for array values.

Part H: Primitive vs Non-Primitive Values

Sprint 8 uses an idea from Sprint 2: objects are non-primitive values.

27. Primitive Values

Primitive values include:

- string
- number
- boolean
- undefined
- null
- symbol
- bigint

Primitive values are copied by value.

Example:

```
let a = 10;  
let b = a;  
  
a = 20;  
  
console.log(a);  
console.log(b);
```

Expected output:

```
20  
10
```

Why?

Because `b` got its own copy of the value `10`.

Changing `a` does not change `b`.

28. Non-Primitive Values

Non-primitive values include:

- objects
- arrays
- functions

Objects are copied by reference.

Example:

```
const user = { name: "Alice" };  
const sameUser = user;  
  
user.name = "Ada";  
  
console.log(sameUser.name);
```

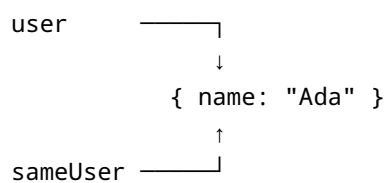
Expected output:

Ada

Why?

Because both variables point to the same object.

Mental model:



Simple meaning:

Objects are not copied as full independent objects by default. Variables usually copy the reference.

29. Why Object References Matter

Object references can surprise beginners.

```
const original = {  
  score: 10  
};  
  
const copy = original;  
  
copy.score = 99;  
  
console.log(original.score);
```

Expected output:

99

The variable name `copy` is misleading here.

It is not a true independent copy.

It points to the same object.

Beginner rule

When you assign one object variable to another, both variables usually point to the same object.

30. `const` With Objects

A `const` object can still be changed inside.

```
const user = {  
  name: "Alice",  
  active: true  
};  
  
user.active = false;  
  
console.log(user);
```

Expected output:

```
{ name: "Alice", active: false }
```

This is allowed because `const` prevents reassignment of the variable.

It does not freeze the object contents.

This is allowed:

```
const user = { name: "Alice" };  
user.name = "Ada";
```

This is not allowed:

```
const user = { name: "Alice" };  
user = { name: "Ada" }; // TypeError
```

Why?

Because this tries to make `user` point to a completely different object.

Beginner rule

`const` protects the variable binding.

`const` does not make the object immutable.

Part I: Methods

31. What Is a Method?

A **method** is a function stored inside an object.

Example:

```
const calculator = {  
  add: function(a, b) {  
    return a + b;  
  }  
};
```

```
console.log(calculator.add(2, 3));
```

Expected output:

```
5
```

The property `add` stores a function.

This:

```
calculator.add(2, 3)
```

means:

Call the `add` function inside the `calculator` object.

32. Short Method Syntax

JavaScript has a shorter way to write methods.

Long version:

```
const calculator = {  
  add: function(a, b) {  
    return a + b;  
  }  
};
```

Short version:

```
const calculator = {  
  add(a, b) {  
    return a + b;  
  }  
};
```

Both versions work.

Example:

```
const calculator = {
  add(a, b) {
    return a + b;
  },
  subtract(a, b) {
    return a - b;
  }
};

console.log(calculator.add(10, 5));
console.log(calculator.subtract(10, 5));
```

Expected output:

```
15
5
```

33. `this` Inside a Method

Inside a beginner object method, `this` often refers to the object that owns the method.

Example:

```
const user = {
  name: "Alice",
  greet() {
    return `Hello, I am ${this.name}`;
  }
};

console.log(user.greet());
```

Expected output:

```
Hello, I am Alice
```

This line:

```
this.name
```


means:

Get the `name` property from this object.

Here, `this` refers to `user` because the method is called as:

```
user.greet()
```

Beginner warning

`this` has more advanced rules in JavaScript.

For Sprint 8, only remember this simple idea:

In a normal object method call, `this` usually refers to the object before the dot.

34. Method Example: Student Average

```
const student = {
  name: "Sangeeth",
  scores: [90, 85, 100],

  getAverageScore() {
    let total = 0;

    for (const score of this.scores) {
      total += score;
    }

    return total / this.scores.length;
  }
};

console.log(student.getAverageScore());
```

Expected output:

```
91.66666666666667
```

Step-by-step:

1. `student` has a `scores` array.

2. `getAverageScore()` is a method.
 3. `this.scores` gets the scores array from the current object.
 4. The loop adds every score.
 5. The method returns the average.
-

Part J: The `Object()` Constructor

35. Object Literal Syntax

The usual beginner-friendly way to create an object is object literal syntax:

```
const user = {  
  name: "Alice",  
  active: true  
};
```

This is clear and common.

36. The `Object()` Constructor

JavaScript also has an `Object()` constructor.

Example:

```
const user = new Object();  
  
user.name = "Alice";  
user.active = true;  
  
console.log(user);
```

Expected output:

```
{ name: "Alice", active: true }
```

This works, but beginner code usually prefers object literal syntax:

```
const user = {  
  name: "Alice",
```

```
    active: true  
};
```

Why?

Because it is shorter, clearer, and easier to read.

37. Passing Values to `Object()`

You may see examples like this:

```
console.log(Object("hello"));  
console.log(Object(42));  
console.log(Object(true));
```

This wraps primitive values into object wrapper forms.

This is not common beginner code.

For normal beginner work, create objects with:

```
const object = {};
```

or:

```
const user = {  
  name: "Alice"  
};
```

38. What Happens With `Object(null)` and `Object(undefined)`?

In modern JavaScript:

```
console.log(Object(null));  
console.log(Object(undefined));
```

These create empty objects.

Expected output:

```
{}  
{}
```

But this is not good beginner style.

It is clearer to write:

```
const user = {};
```

Beginner rule

Avoid using `Object(null)` or `Object(undefined)` in beginner code.

Use object literal syntax instead.

Part K: JSON

39. What Is JSON?

JSON stands for **JavaScript Object Notation**.

JSON is a text format used to store or exchange data.

It looks similar to JavaScript objects, but it is not the same thing.

JavaScript object:

```
const user = {  
  name: "Alice",  
  active: true  
};
```

JSON string:

```
const json = '{"name":"Alice","active":true}';
```

Simple meaning:

JSON is data written as text.

JSON is often used when:

- sending data to a server
 - receiving data from an API
 - saving data in browser storage
 - storing settings
 - exchanging structured data between systems
-

40. JSON Syntax Rules

JSON has stricter rules than JavaScript object literals.

Valid JSON:

```
{  
  "name": "Alice",  
  "active": true,  
  "score": 95,  
  "courses": ["HTML", "CSS", "JavaScript"],  
  "profile": null  
}
```

Important JSON rules:

1. Property names must use double quotes.
 2. String values must use double quotes.
 3. JSON can store objects.
 4. JSON can store arrays.
 5. JSON can store strings.
 6. JSON can store numbers.
 7. JSON can store booleans.
 8. JSON can store `null`.
 9. JSON cannot store functions.
 10. JSON cannot contain comments.
 11. JSON cannot use trailing commas.
-

41. JavaScript Object vs JSON

JavaScript object:

```
const user = {
  name: "Alice",
  active: true,
  greet() {
    return "Hello";
  }
};
```

JSON:

```
{
  "name": "Alice",
  "active": true
}
```

Main differences:

Feature	JavaScript Object	JSON
Type	Live JavaScript value	Text format
Keys	Quotes optional for simple keys	Double quotes required
Strings	Single quotes, double quotes, or backticks	Double quotes only
Functions	Allowed as methods	Not allowed
Comments	Possible outside object syntax	Not allowed
Trailing commas	Sometimes allowed in JS	Not allowed

Beginner rule

Do not confuse a JavaScript object with JSON text.

They look similar, but they are used differently.

42. `JSON.stringify()`

`JSON.stringify()` converts a JavaScript value into a JSON string.

Example:

```
const user = {
  name: "Alice",
  active: true
};

const json = JSON.stringify(user);

console.log(json);
console.log(typeof json);
```

Expected output:

```
{"name":"Alice","active":true}
string
```

Simple meaning:

Convert the object into text.

Use `JSON.stringify()` when you want to save or send data.

43. `JSON.parse()`

`JSON.parse()` converts JSON text back into a JavaScript value.

Example:

```
const json = '{"name":"Alice","active":true}';

const user = JSON.parse(json);

console.log(user.name);
console.log(user.active);
console.log(typeof user);
```

Expected output:

```
Alice
true
object
```

Simple meaning:

Convert JSON text back into a JavaScript object.

Use `JSON.parse()` when you load or receive JSON text.

44. JSON Round Trip

A round trip means:

```
object -> JSON string -> object
```

Example:

```
const originalUser = {
  name: "Alice",
  score: 95,
  active: true
};

const savedText = JSON.stringify(originalUser);
const loadedUser = JSON.parse(savedText);

console.log(savedText);
console.log(loadedUser.name);
console.log(loadedUser.score);
console.log(loadedUser.active);
```

Expected output:

```
{"name":"Alice","score":95,"active":true}
Alice
95
true
```

Mental model:

```
object -> JSON.stringify() -> string
string -> JSON.parse() -> object
```

45. `JSON.stringify()` With Spacing

`JSON.stringify()` can format JSON with indentation.

```
const user = {
  name: "Alice",
  active: true,
  score: 95
};

const prettyJson = JSON.stringify(user, null, 2);

console.log(prettyJson);
```

Expected output:

```
{
  "name": "Alice",
  "active": true,
  "score": 95
}
```

The third argument controls spacing:

```
JSON.stringify(user, null, 2)
```

- `user` is the value to convert.
- `null` means no custom replacer.
- `2` means use two spaces for indentation.

This is useful when you want JSON to be easier to read.

46. `JSON.stringify()` With a Replacer

A replacer can control what gets included.

Example:

```
const user = {
  name: "Alice",
  password: "secret123",
```

```
    role: "student"
  };

const safeJson = JSON.stringify(user, ["name", "role"], 2);

console.log(safeJson);
```

Expected output:

```
{
  "name": "Alice",
  "role": "student"
}
```

The array:

```
["name", "role"]
```

means:

Only include these properties.

Beginner note:

You do not need to master replacers immediately, but you should know that `JSON.stringify()` can accept extra options.

47. Invalid JSON Causes an Error

`JSON.parse()` expects valid JSON.

This works:

```
const json = '{"name":"Alice"}';
const user = JSON.parse(json);
console.log(user.name);
```

This fails:

```
const badJson = '{name:"Alice"}';  
const user = JSON.parse(badJson);
```

Why?

Because JSON keys must use double quotes.

Correct JSON:

```
{"name": "Alice"}
```

Incorrect JSON:

```
{name: "Alice"}
```

Beginner rule

If `JSON.parse()` fails, check whether the JSON string is valid.

Part L: Optional Chaining

48. The Problem Optional Chaining Solves

Sometimes data is missing.

Example:

```
const user = {  
  name: "Alice"  
};  
  
console.log(user.profile.email);
```

This crashes because `profile` does not exist.

JavaScript tries to read:

```
undefined.email
```

That causes an error.

49. Optional Chaining `?.`

Optional chaining safely stops if a value is `null` or `undefined`.

Example:

```
const user = {  
  name: "Alice"  
};  
  
console.log(user.profile?.email);
```

Expected output:

```
undefined
```

No crash.

Simple meaning:

If `profile` exists, get `email`. If `profile` is missing, return `undefined`.

50. Optional Chaining With Deeper Paths

```
const user = {  
  name: "Alice"  
};  
  
console.log(user.profile?.contact?.email);
```

Expected output:

```
undefined
```

This safely checks each optional part.

Without optional chaining:

```
console.log(user.profile.contact.email); // TypeError
```

With optional chaining:

```
console.log(user.profile?.contact?.email); // undefined
```

51. Optional Chaining With Arrays Inside Objects

```
const student = {  
  name: "Sangeeth",  
  courses: [  
    { title: "HTML" },  
    { title: "JavaScript" }  
  ]  
};  
  
console.log(student.courses?.[1]?.title);  
console.log(student.courses?.[5]?.title);
```

Expected output:

```
JavaScript  
undefined
```

This line:

```
student.courses?.[5]?.title
```

means:

If `courses` exists, try index `5`. If that result exists, get `title`. Otherwise return `undefined`.

52. When to Use Optional Chaining

Use optional chaining when:

- data may be missing

- data comes from an API
- a user may not have completed a profile
- a nested object may not exist
- you want to avoid crashes while reading optional data

Example:

```
const city = user.address?.city;
```

Simple meaning:

Get the city if the address exists.

Beginner warning

Optional chaining prevents crashes while reading data.

It does not create missing data.

```
const user = {};  
  
console.log(user.profile?.email); // undefined
```

This does not create `profile` or `email`.

Part M: Object Destructuring

53. What Is Object Destructuring?

Object destructuring lets you extract properties into variables.

Example:

```
const user = {  
  name: "Alice",  
  age: 25  
};  
  
const { name, age } = user;
```

```
console.log(name);  
console.log(age);
```

Expected output:

```
Alice  
25
```

This line:

```
const { name, age } = user;
```

means:

Create variables named `name` and `age` from matching properties in `user`.

54. Destructuring Depends on Property Names

Object destructuring uses property names, not positions.

Example:

```
const user = {  
  age: 25,  
  name: "Alice"  
};  
  
const { name, age } = user;  
  
console.log(name);  
console.log(age);
```

Expected output:

```
Alice  
25
```

Even though `age` came first in the object, destructuring still finds the correct property by name.

This is different from array destructuring, which uses position.

55. Destructuring With Default Values

You can provide a default value if the property is missing.

```
const user = {  
  name: "Alice"  
};  
  
const { name, country = "Unknown" } = user;  
  
console.log(name);  
console.log(country);
```

Expected output:

```
Alice  
Unknown
```

Why?

The object has no `country` property, so JavaScript uses the default value.

56. Destructuring With Renaming

You can extract a property but store it in a variable with a different name.

```
const user = {  
  name: "Alice"  
};  
  
const { name: userName } = user;  
  
console.log(userName);
```

Expected output:

```
Alice
```

This line:


```
const { name: userName } = user;
```

means:

Take `user.name`, but store it in a variable called `userName`.

Important:

```
console.log(name); // ReferenceError
```

There is no variable named `name` in this example.

The variable is named `userName`.

57. Destructuring With Renaming and Default Values

You can combine renaming and default values.

```
const user = {  
  name: "Alice"  
};  
  
const { country: userCountry = "Unknown" } = user;  
  
console.log(userCountry);
```

Expected output:

```
Unknown
```

This means:

Take `country` from `user`. If it is missing, use `"Unknown"`. Store the result in `userCountry`.

58. Nested Destructuring

You can destructure nested objects.

```
const user = {
  name: "Alice",
  contact: {
    email: "alice@example.com"
  }
};

const {
  contact: { email }
} = user;

console.log(email);
```

Expected output:

```
alice@example.com
```

This means:

Go into `contact`, then extract `email`.

Beginner warning

Nested destructuring can crash if the nested object is missing.

Example:

```
const user = {
  name: "Alice"
};

const {
  contact: { email }
} = user; // TypeError
```

Why?

Because `contact` does not exist.

Safer beginner version:

```
const email = user.contact?.email;  
console.log(email);
```

Expected output:

```
undefined
```

59. Object Rest Syntax

Object rest syntax collects the remaining properties into a new object.

```
const user = {  
  name: "Alice",  
  age: 25,  
  role: "student"  
};  
  
const { name, ...details } = user;  
  
console.log(name);  
console.log(details);
```

Expected output:

```
Alice  
{ age: 25, role: "student" }
```

This line:

```
const { name, ...details } = user;
```

means:

Take `name` out. Put the rest into `details`.

Part N: Object Property Shorthand

60. What Is Property Shorthand?

When the variable name and property name are the same, you can use shorthand.

Long version:

```
const name = "Alice";
const age = 25;

const user = {
  name: name,
  age: age
};

console.log(user);
```

Short version:

```
const name = "Alice";
const age = 25;

const user = {
  name,
  age
};

console.log(user);
```

Both create:

```
{ name: "Alice", age: 25 }
```

Simple meaning:

If the key and variable name are the same, you only need to write the name once.

61. Property Shorthand in Functions

Property shorthand is useful when returning objects from functions.

```
function createUser(name, age) {  
  return {  
    name,  
    age,  
    active: true  
  };  
}  
  
const user = createUser("Alice", 25);  
console.log(user);
```

Expected output:

```
{ name: "Alice", age: 25, active: true }
```

Without shorthand, you would write:

```
function createUser(name, age) {  
  return {  
    name: name,  
    age: age,  
    active: true  
  };  
}
```

The shorthand version is cleaner.

Part O: Core Sprint 8 Code Pattern

Type this code yourself.

Before running it, predict the output.

```
const person = {  
  name: "Alice",  
  contact: {  
    phone: {  
      work: "098-765-4321"  
    }  
  },  
  addresses: [  

```

```
    { city: "Homeville" },  
    { city: "Workville" }  
  ]  
};  
  
console.log(person.contact.phone.work);  
console.log(person.addresses[1].city);  
console.log(person.profile?.email);  
  
const { name, country = "Unknown" } = person;  
  
console.log(name);  
console.log(country);
```

Expected output:

```
098-765-4321  
Workville  
undefined  
Alice  
Unknown
```

62. Core Pattern Explanation

This object has a nested contact structure:

```
contact: {  
  phone: {  
    work: "098-765-4321"  
  }  
}
```

So this:

```
person.contact.phone.work
```

returns:

```
098-765-4321
```

This object also has an array of address objects:

```
addresses: [  
  { city: "Homeville" },  
  { city: "Workville" }  
]
```

So this:

```
person.addresses[1].city
```

returns:

```
Workville
```

This line:

```
person.profile?.email
```

returns:

```
undefined
```

because `profile` does not exist, but optional chaining prevents a crash.

This line:

```
const { name, country = "Unknown" } = person;
```

extracts `name` and `country`.

`name` exists, so it becomes:

```
Alice
```

`country` does not exist, so it uses the default value:

Unknown

Part P: Practical Sprint 8 Examples

63. Example: Student Profile Object

```
const student = {
  id: 1,
  name: "Sangeeth",
  active: true,
  scores: [90, 85, 100],
  contact: {
    email: "sangeeth@example.com",
    phone: null
  },
  getAverageScore() {
    let total = 0;

    for (const score of this.scores) {
      total += score;
    }

    return total / this.scores.length;
  }
};

console.log(student.name);
console.log(student.contact.email);
console.log(student.contact.phone);
console.log(student.contact.address?.city);
console.log(student.getAverageScore());
```

Expected output:

```
Sangeeth
sangeeth@example.com
null
undefined
91.66666666666667
```


Concepts used:

- object properties
 - nested objects
 - arrays inside objects
 - a method
 - `this`
 - a loop inside a method
 - optional chaining
 - `null` for intentional emptiness
-

64. Example: Book Object

```
const book = {  
  title: "JavaScript Basics",  
  author: "Code Teacher",  
  pages: 250,  
  isFinished: false,  
  tags: ["programming", "beginner", "javascript"],  
  publisher: {  
    name: "Learning Press",  
    country: "USA"  
  }  
};  
  
console.log(book.title);  
console.log(book.tags[0]);  
console.log(book.publisher.name);
```

Expected output:

```
JavaScript Basics  
programming  
Learning Press
```

65. Example: Add and Update Book Data

```
const book = {  
  title: "JavaScript Basics",  
  isFinished: false  
};
```

```
book.rating = 5;
book.isFinished = true;

console.log(book);
```

Expected output:

```
{ title: "JavaScript Basics", isFinished: true, rating: 5 }
```

What happened?

```
book.rating = 5;
```

added a new property.

```
book.isFinished = true;
```

updated an existing property.

66. Example: Safe Property Access

Risky version:

```
const book = {
  title: "JavaScript Basics"
};

console.log(book.reviews.firstReview.text);
```

This crashes because `reviews` does not exist.

Safe version:

```
const book = {
  title: "JavaScript Basics"
};
```

```
console.log(book.reviews?.firstReview?.text);
```

Expected output:

```
undefined
```

Optional chaining prevents the crash.

67. Example: Convert Book to JSON and Back

```
const book = {  
  title: "JavaScript Basics",  
  pages: 250,  
  isFinished: false  
};  
  
const bookJson = JSON.stringify(book);  
const parsedBook = JSON.parse(bookJson);  
  
console.log(bookJson);  
console.log(parsedBook.title);
```

Expected output:

```
{"title":"JavaScript Basics","pages":250,"isFinished":false}  
JavaScript Basics
```

Part Q: Common Beginner Mistakes

68. Mistake 1: Using Dot Notation for a Dynamic Key

Wrong:

```
const user = {  
  name: "Alice"  
};
```

```
const key = "name";  
  
console.log(user.key);
```

Expected output:

```
undefined
```

Why?

`user.key` looks for a property literally named `key`.

Correct:

```
console.log(user[key]);
```

Expected output:

```
Alice
```

69. Mistake 2: Thinking `const` Freezes an Object

Wrong idea:

```
const user = { name: "Alice" };  
user.name = "Ada"; // This is allowed.
```

`const` does not freeze the object.

It only prevents reassignment.

Not allowed:

```
user = { name: "Grace" }; // TypeError
```

70. Mistake 3: Crashing on Missing Nested Data

Risky:

```
console.log(user.profile.email);
```

Safer:

```
console.log(user.profile?.email);
```

Optional chaining returns `undefined` instead of crashing when `profile` is missing.

71. Mistake 4: Confusing JSON With JavaScript Objects

JavaScript object:

```
const user = { name: "Alice" };
```

JSON text:

```
const json = '{"name":"Alice"}';
```

A JavaScript object is a live value in memory.

JSON is text.

72. Mistake 5: Checking Properties With Truthy/Falsy Logic

Risky:

```
if (user.score) {  
  console.log("Score exists");  
}
```

This fails when `score` is `0`.

Safer:

```
if (Object.hasOwn(user, "score")) {  
  console.log("Score exists");  
}
```

73. Mistake 6: Forgetting That JSON Does Not Store Functions

Example:

```
const user = {  
  name: "Alice",  
  greet() {  
    return "Hello";  
  }  
};  
  
const json = JSON.stringify(user);  
console.log(json);
```

Expected output:

```
{"name": "Alice"}
```

The method is not included in the JSON string.

Beginner rule

JSON stores data, not behavior.

74. Mistake 7: Using Nested Destructuring on Missing Data

Risky:

```
const user = {  
  name: "Alice"  
};  
  
const {  
  name,  
  greet,  
  ...rest  
}
```

```
    contact: { email }  
  } = user;
```

This crashes because `contact` does not exist.

Safer:

```
const email = user.contact?.email;  
console.log(email);
```

Expected output:

```
undefined
```

Part R: Practice Tasks

75. Task 1: Create a User Object

Create a `user` object with these properties:

- `name`
- `age`
- `email`
- `isActive`
- `score`

Example shape:

```
const user = {  
  name: "Sangeeth",  
  age: 16,  
  email: "sangeeth@example.com",  
  isActive: true,  
  score: 0  
};
```

Then print:

```
user.name  
user.email  
user.score
```

76. Task 2: Add and Update Properties

Start with this object:

```
const user = {  
  name: "Alice",  
  score: 10  
};
```

Add:

```
user.active = true;
```

Update:

```
user.score = 20;
```

Print the final object.

77. Task 3: Use Bracket Notation

Create this object:

```
const course = {  
  title: "JavaScript",  
  "course level": "beginner"  
};
```

Print the `"course level"` property using bracket notation.

Expected result:

beginner

78. Task 4: Dynamic Property Access

```
const user = {  
  name: "Alice",  
  email: "alice@example.com"  
};  
  
const key = "email";
```

Use `key` to print the email.

Expected result:

alice@example.com

79. Task 5: Property Existence Check

```
const player = {  
  name: "Ada",  
  score: 0  
};
```

Check whether `score` exists using `Object.hasOwnProperty()`.

Expected result:

true

Do not use this:

```
if (player.score)
```

because `0` is falsy.

80. Task 6: Nested Object

Create a `profile` object with nested contact information.

Example:

```
const profile = {  
  username: "coder123",  
  contact: {  
    email: "coder@example.com",  
    phone: {  
      work: "111-222-3333"  
    }  
  }  
};
```

Print:

```
profile.contact.email  
profile.contact.phone.work
```

81. Task 7: Optional Chaining

Start with:

```
const profile = {  
  username: "coder123"  
};
```

Try to safely read:

```
profile.contact.email
```

using optional chaining.

Correct pattern:

```
console.log(profile.contact?.email);
```

Expected result:

undefined

82. Task 8: Array of Objects

Create an array of three task objects.

Each task should have:

- id
- title
- done

Example:

```
const tasks = [  
  { id: 1, title: "Study objects", done: true },  
  { id: 2, title: "Practice JSON", done: false },  
  { id: 3, title: "Build mini project", done: false }  
];
```

Loop through the tasks and print each title.

```
for (const task of tasks) {  
  console.log(task.title);  
}
```

83. Task 9: Method Practice

Create a `calculator` object with methods:

- `add(a, b)`
- `subtract(a, b)`
- `multiply(a, b)`

Example:

```
const calculator = {
  add(a, b) {
    return a + b;
  },
  subtract(a, b) {
    return a - b;
  },
  multiply(a, b) {
    return a * b;
  }
};
```

Print the result of each method.

84. Task 10: JSON Practice

Create an object:

```
const book = {
  title: "JavaScript Basics",
  pages: 250,
  finished: false
};
```

Convert it to JSON:

```
const bookJson = JSON.stringify(book);
```

Convert it back:

```
const parsedBook = JSON.parse(bookJson);
```

Print:

```
parsedBook.title
```

Part S: Mini-Builds

85. Mini-Build 1: Student Report

Goal:

Create a student object that stores scores and calculates an average.

```
const student = {  
  name: "Sangeeth",  
  scores: [90, 85, 100],  
  
  getAverage() {  
    let total = 0;  
  
    for (const score of this.scores) {  
      total += score;  
    }  
  
    return total / this.scores.length;  
  }  
};  
  
console.log(`${student.name}'s average is ${student.getAverage()}`);
```

Expected output:

```
Sangeeth's average is 91.66666666666667
```

Concepts used:

- object
- array inside object
- method
- `this`
- loop
- return value
- template literal

86. Mini-Build 2: Safe User Summary

Goal:

Create a safe summary from a user object without exposing the password.

```
const user = {
  name: "Alice",
  email: "alice@example.com",
  password: "secret123",
  role: "student"
};

const { password, ...safeUser } = user;

console.log(safeUser);
```

Expected output:

```
{
  name: "Alice",
  email: "alice@example.com",
  role: "student"
}
```

Concepts used:

- destructuring
- object rest syntax
- creating a new object without one property

87. Mini-Build 3: Course Data to JSON

Goal:

Create a course object, convert it to JSON, then parse it back.

```
const course = {
  title: "JavaScript Sprint 8",
  level: "beginner",
  topics: ["objects", "methods", "JSON", "optional chaining"]
};

const json = JSON.stringify(course, null, 2);
const parsedCourse = JSON.parse(json);
```

```
console.log(json);  
console.log(parsedCourse.topics[2]);
```

Expected output:

```
{  
  "title": "JavaScript Sprint 8",  
  "level": "beginner",  
  "topics": [  
    "objects",  
    "methods",  
    "JSON",  
    "optional chaining"  
  ]  
}  
JSON
```

Concepts used:

- object
- array inside object
- JSON formatting
- `JSON.stringify()`
- `JSON.parse()`
- nested access

Part T: Sprint 8 Revision Checklist

You should be able to explain each item clearly.

Object Basics

- I know what an object is.
- I know what a property is.
- I know what a key-value pair is.
- I can create an object literal.
- I can store strings, numbers, booleans, arrays, and objects inside an object.

Property Access

- I can use dot notation.
- I can use bracket notation.
- I know when bracket notation is required.

- I can use a variable as a dynamic property key.
- I understand why `object.key` and `object[key]` are different.

Property Changes

- I can add a new property.
- I can update an existing property.
- I can delete a property with `delete`.
- I can create a new object without a property using rest syntax.

Property Checks

- I know why `if (user.score)` can be wrong.
- I know that `0`, `false`, and `""` can be valid values.
- I can use `Object.hasOwnProperty()` to check if a property exists.
- I know the basic difference between `Object.hasOwnProperty()` and the `in` operator.

Nested Data

- I can read nested object properties.
- I can read arrays inside objects.
- I can read objects inside arrays.
- I can combine object access and array indexing.

References and `const`

- I know that objects are copied by reference.
- I know that `const` does not freeze an object.
- I know the difference between changing an object property and reassigning the object variable.

Methods

- I know what a method is.
- I can create a method inside an object.
- I can call a method with dot notation.
- I understand the beginner meaning of `this` inside a normal object method.

JSON

- I know what JSON is.
- I know JSON is text, not a live JavaScript object.
- I know JSON keys must use double quotes.
- I can use `JSON.stringify()`.
- I can use `JSON.parse()`.
- I know JSON cannot store functions.

Optional Chaining

- I can use `?.` to safely access missing nested data.
- I know optional chaining returns `undefined` instead of crashing when a property before it is `null` or `undefined`.
- I know optional chaining does not create missing data.

Destructuring and Shorthand

- I can destructure object properties into variables.
- I can rename variables during destructuring.
- I can use default values during destructuring.
- I can use nested destructuring carefully.
- I can use object rest syntax.
- I can use property shorthand.

Part U: Sprint 8 Checkpoint Questions

Answer these without looking at the notes.

1. What is an object?
2. What is a property?
3. What is a key-value pair?
4. When should you use dot notation?
5. When should you use bracket notation?
6. What is dynamic property access?
7. Why does `user.key` not use the value inside the `key` variable?
8. How do you add a new property to an object?
9. How do you update an existing property?
10. What does `delete` do?
11. How can you create a new object without one selected property?
12. Why can `if (user.score)` be wrong when `score` is `0`?
13. What does `Object.hasOwnProperty(user, "score")` check?
14. What is the difference between `Object.hasOwnProperty()` and the `in` operator?
15. What is a nested object?
16. How do you access an object inside an array?
17. How do you access an array inside an object?
18. What does it mean that objects are copied by reference?
19. Why can a `const` object still have its properties changed?
20. What is a method?
21. What does `this` usually mean inside a normal object method call?
22. What is JSON?
23. Why is JSON not exactly the same as a JavaScript object?
24. What does `JSON.stringify()` do?

25. What does `JSON.parse()` do?
 26. What does optional chaining prevent?
 27. What does `user.profile?.email` return if `profile` is missing?
 28. What does object destructuring do?
 29. How do you rename a variable during destructuring?
 30. What is object property shorthand?
-

Part V: Minimum Passing Standard

You are ready to move on from Sprint 8 when you can explain every line of this code.

```
const user = {
  name: "Alice",
  score: 0,
  contact: {
    email: "alice@example.com"
  },
  courses: [
    { title: "HTML", complete: true },
    { title: "JavaScript", complete: false }
  ],

  getSummary() {
    return `${this.name} has ${this.courses.length} courses.`;
  }
};

console.log(user.name);
console.log(user.contact?.email);
console.log(user.profile?.city);
console.log(user.courses[1].title);
console.log(Object.hasOwn(user, "score"));
console.log(user.getSummary());

const { name, country = "Unknown" } = user;
console.log(name);
console.log(country);

const json = JSON.stringify(user, null, 2);
const parsed = JSON.parse(json);

console.log(parsed.name);
```

Expected output:

```
Alice
alice@example.com
undefined
JavaScript
true
Alice has 2 courses.
Alice
Unknown
Alice
```

Important note:

The JSON output itself also appears in the console because of this line:

```
const json = JSON.stringify(user, null, 2);
```

But the method `getSummary()` is not stored in the JSON because JSON does not store functions.

Part W: Final Sprint 8 Summary

Sprint 8 teaches you how to organize data like a real JavaScript program.

The most important ideas are:

1. Objects store related data as key-value pairs.
2. Dot notation is for simple known property names.
3. Bracket notation is for dynamic keys or unusual property names.
4. Objects can contain other objects and arrays.
5. Arrays can contain objects.
6. Methods are functions stored inside objects.
7. Objects are copied by reference.
8. `const` does not freeze object contents.
9. Property existence checks should not rely only on truthy/falsy values.
10. `Object.hasOwnProperty()` is a safe beginner-friendly property existence check.
11. Optional chaining prevents crashes when reading missing nested data.
12. JSON is a text format for storing or exchanging data.
13. `JSON.stringify()` turns JavaScript data into JSON text.
14. `JSON.parse()` turns JSON text back into JavaScript data.
15. Object destructuring extracts properties into variables.
16. Property shorthand makes object creation cleaner when variable names and property names match.

If you understand these ideas, you can now represent structured data like users, books, tasks, courses, products, profiles, and API responses.